

Hadoop与Spark

Zookeeper Misc.

https://blog.51cto.com/u_15724848/5766927

https://blog.csdn.net/qq_44750696/article/details/123759164

Zookeeper Misc.

◆ Zookeeper是什么?

- ◆ Zookeeper是一个开放源码的、分布式的应用程序协调服务，它是Google的Chubby一个开源的实现，是Hadoop和HBase的重要组件
- ◆ 它是一个为分布式应用提供一致性服务的软件
- ◆ 作为开源的分布式协调框架，将那些复杂且容易出错的分布式一致性服务封装起来，构成一个高效可靠的原语集，并以一系列简单易用的接口提供给用户使用

A Distributed Coordination Service for Distributed Applications



Apache ZooKeeper™

Zookeeper Misc.

- ◆ **Zookeeper能干什么?**
 - ◆ Zookeeper是Hadoop下的一个子项目
 - ◆ 它是一个针对大型分布式系统的可靠协调系统
 - ◆ 提供的功能包括：配置维护、名字服务、分布式同步服务、组服务等
 - ◆ 分布式应用程序可以基于Zookeeper实现诸如数据发布/订阅、负载均衡、命名服务、集群管理、Master 选举、分布式锁、分布式队列等功能...



Apache ZooKeeper™

pause



Zookeeper Misc.

- ◆ Zookeeper
 - ◆ ZK之所以能够提供上述一套分布式数据一致性解决方案
 - ◆ 核心在于其设计精妙的数据结构、watcher机制、Zab一致性协议等

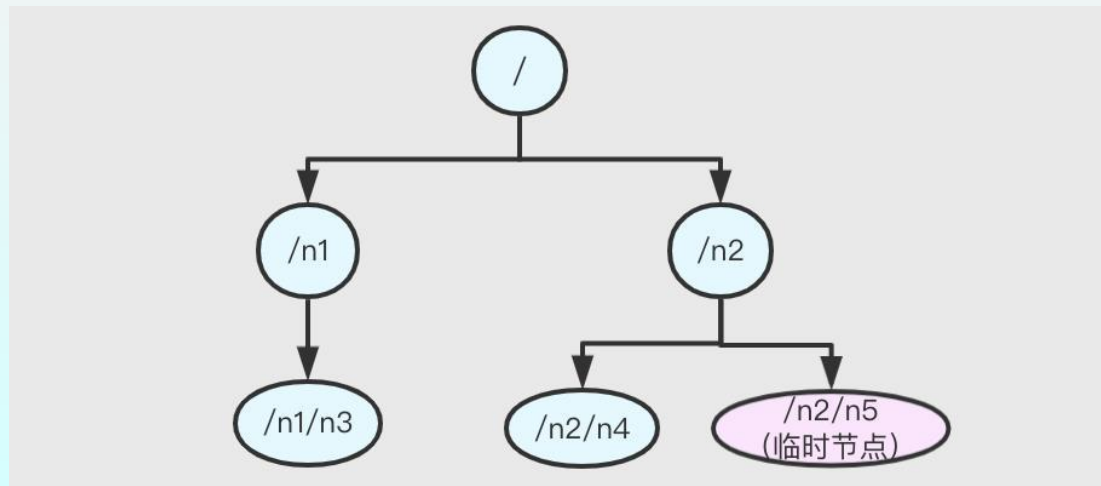
Zookeeper Misc.

- ◆ Zookeeper
 - ◆ Zookeeper allows distributed processes to coordinate with each other through a **shared hierarchical namespace** which is organized similarly to a standard file system
 - ◆ **The namespace consists of data registers - called znodes**, in Zookeeper parlance - and these are similar to files and directories
 - ◆ The Zookeeper implementation puts a premium on high performance, highly available, strictly ordered access.

Zookeeper Misc.

◆ Zookeeper的znode

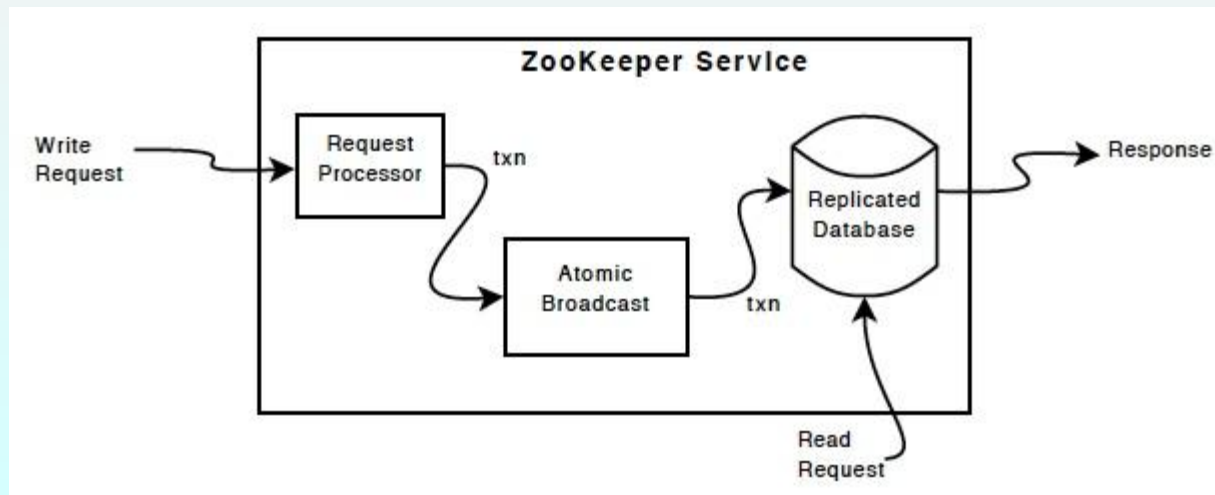
- ◆ ZK在内存中维护了一个类似文件系统的树状数据结构实现命名空间，树中的节点称为 znode
- ◆ 每个znode都对应一个唯一的路径
- ◆ **znode的数据操作具有原子性**，读操作将获取与节点相关的所有数据，写操作也将替换掉节点的所有数据
- ◆ znode可存储的最大数据量是1MB，但实际上我们在znode的数据量应该尽可能小，因为数据过大会导致Zookeeper的性能明显下降



Zookeeper Misc.

❖ Zookeeper Implementation

- ❖ shows the high-level components of the Zookeeper service.
- ❖ The replicated database is an in-memory database containing the entire data tree. **Updates are logged to disk for recoverability, and writes are serialized to disk before they are applied to the in-memory database.**
- ❖ As part of the agreement protocol **all write requests from clients are forwarded to a single server, called the leader.** The rest of the Zookeeper servers, called followers, receive message proposals from the leader and agree upon message delivery. The messaging layer takes care of replacing leaders on failures and syncing followers with leaders.
- ❖ ZooKeeper uses a custom **atomic messaging protocol.**

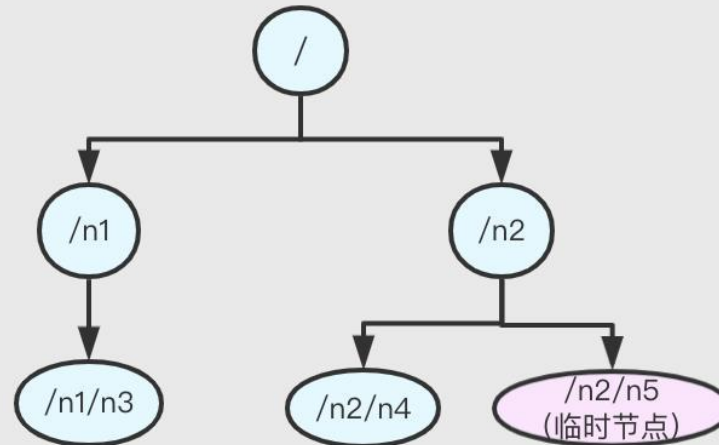


Zookeeper Misc.

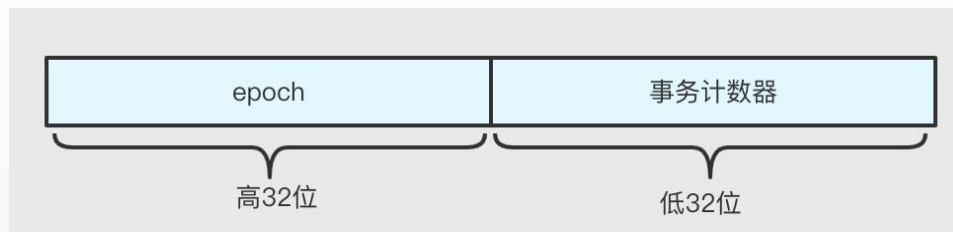
◆ Zookeeper的znode

◆ znode具有四个属性data、acl、stat、children

- ◆ data: znode相关的业务数据均存储在这里
- ◆ children: 存储当前节点的子节点引用信息
- ◆ stat: 包含znode节点的状态信息, 比如: 事务id、版本号、时间戳等, 其中事务id和ZK的数据一致性、选主相关
 - ◆ Each time a znode's data changes, the version number increases
- ◆ acl: 记录每个znode对znode所占的访问权限



Zookeeper Misc.



◆ Zookeeper的znode

- ◆ 事物ID: Zxid

- ◆ **Zxid由Leader节点生成**；当有新写入事件时，Leader节点生成新的Zxid，并随提案一起广播

- ◆ - epoch: 任期/纪元，Zxid的高32位，ZAB协议通过epoch编号来区分Leader周期变化，每次一个leader被选出来，它都会有一个新的 epoch = (原来的epoch + 1)，标识当前属于那个leader的统治时期
- ◆ - 事务计数器: Zxid的低32位，每次数据变更，计数器都会加一
- ◆ zxid是递增的，所以谁的zxid越大，就表示谁的数据是最新的；每个节点都保存了当前最近一次事务的Zxid；Zxid对于ZK的数据一致性以及选主都有着重要意义

Zookeeper Misc.

- ◆ Zookeeper的znode

- ◆ **znode类型**

- ◆ 节点根据生命周期的不同可以分为**持久节点**和**临时节点**
 - ◆ 持久节点的存活时间不依赖于客户端会话，只有客户端在显式执行删除节点操作时，节点才消失；临时节点的存活时间依赖于客户端会话，当会话结束，临时节点将会被自动删除（当然也可以手动删除临时节点）
 - ◆ **create**命令还有一个可选参数-s 用于指定创建的节点是否具有顺序特性
 - ◆ 创建顺序节点时，**zk会在路径后面自动追加一个递增的序列号**，这个序列号可以保证在同一个父节点下是唯一的，利用该特性我们可以实现分布式锁等功能

pause



Zookeeper Misc.

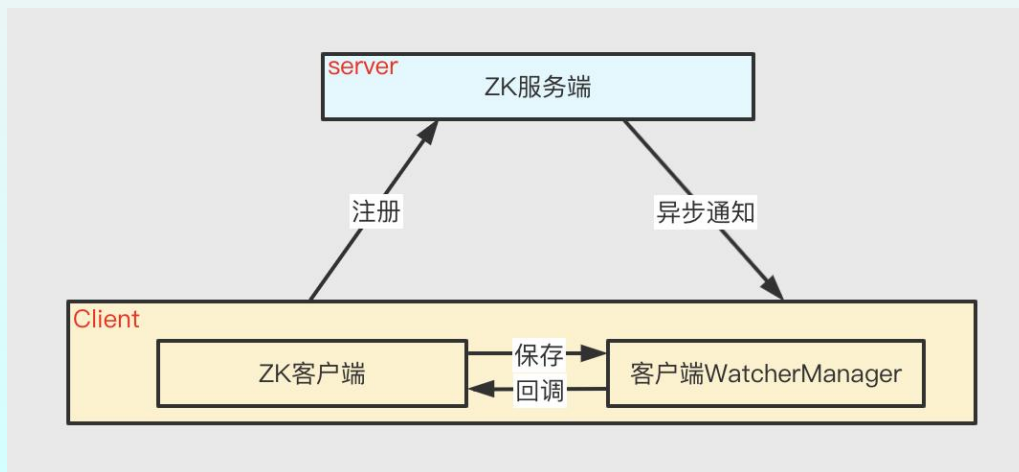
◆ **Watcher** 监听机制

- ◆ Watcher 监听机制是ZK非常重要的一个特性；ZK允许Client端在指定节点上注册Watcher，监听节点数据变更、节点删除、子节点状态变更等事件
- ◆ 当特定事件发生时，ZK服务端会异步通知注册了相应Watcher的客户端，通过该机制，我们可以利用ZK实现**数据的发布和订阅功能**

Zookeeper Misc.

◆ Watcher监听机制

- ◆ Watcher监听机制由三部分协作完成：ZK服务端、ZK客户端、客户端的Watch Manager对象
- ◆ 工作时，客户端首先将Watcher 注册到服务端，同时将Watcher对象保存到客户端的Watch管理器中
- ◆ 当ZK服务端监听的数据状态发生变化时，服务端会主动通知客户端，接着客户端的Watch管理器会触发相关Watcher来回调相应处理逻辑，从而完成整体的数据发布/订阅流程



pause



Zookeeper Misc.

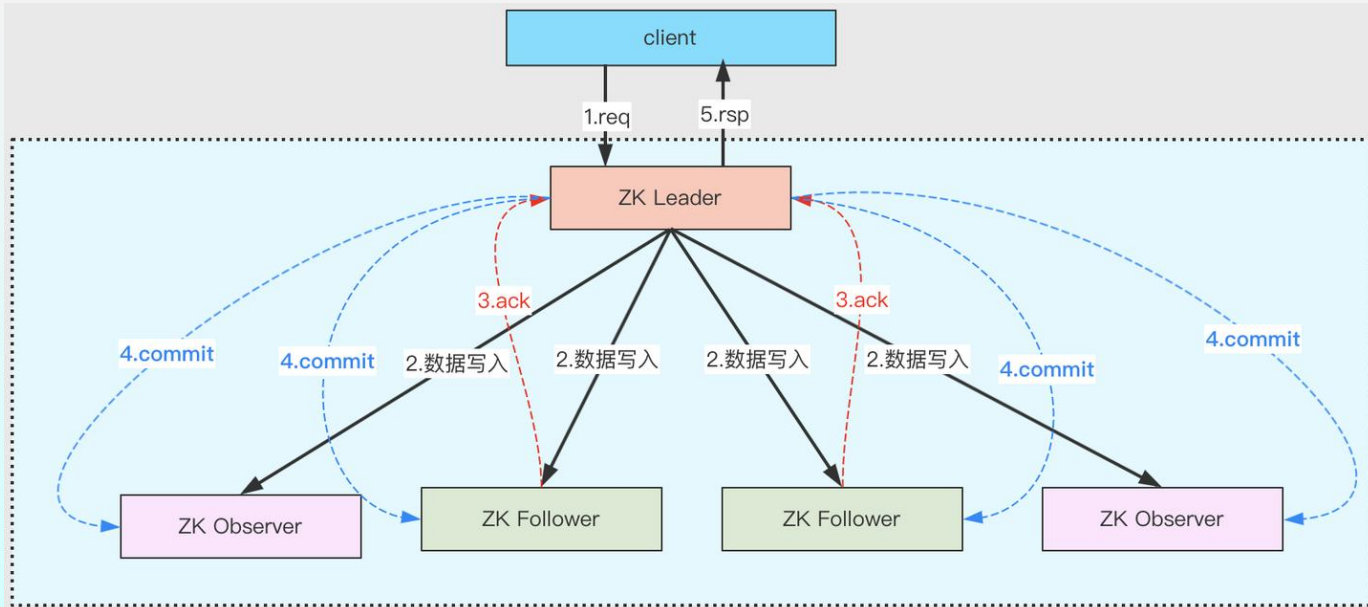
◆ 一致性

- ◆ ZK集群是一主多从的结构，所有的所有的写操作必须要通过 Leader 完成，Follower 可直接处理并返回客户端的读请求
 - ◆ 那么如何保证从Follower服务器读取的数据与Leader写入的数据的一致性呢？
 - ◆ Leader 是怎么选出来的？
 - ◆ Leader万一由于某些原因崩溃了，如何选出新的Leader，如何保证数据恢复？
- ◆ ZK专门设计了ZAB协议(Zookeeper Atomic Broadcast)来保证主从节点数据的一致性

Zookeeper Misc.

◆ 一致性：写 Leader 场景数据一致性

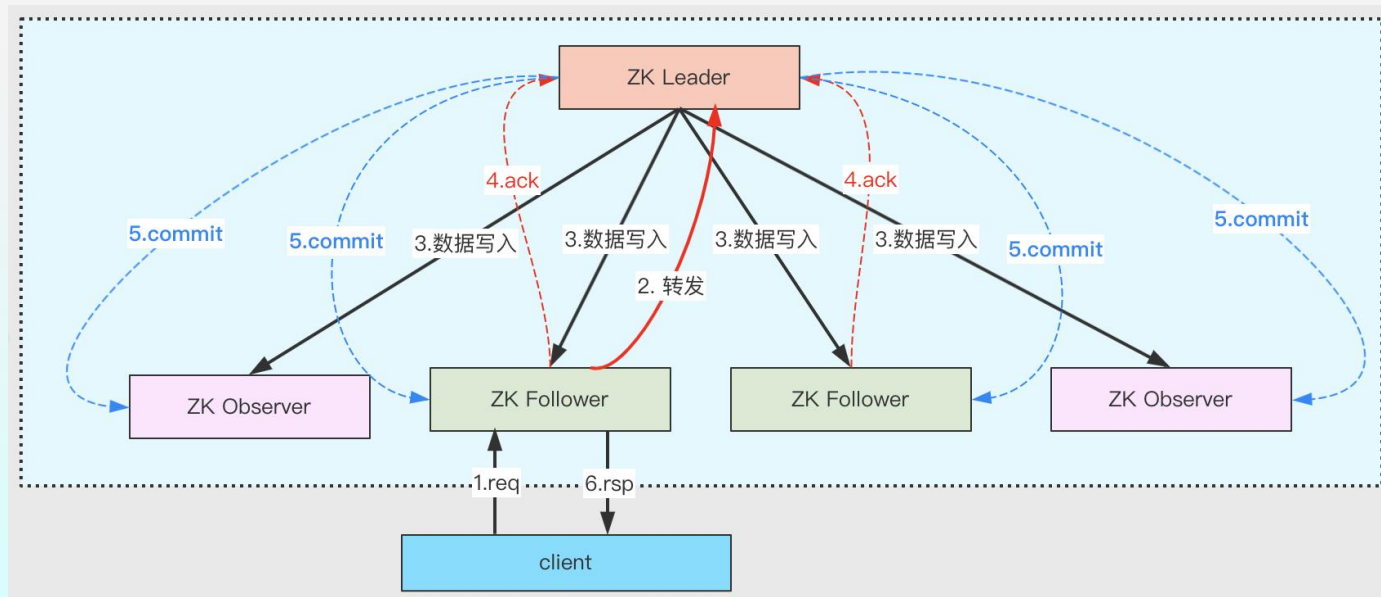
- ◆ 1. 客户端向 Leader 发起写请求
- ◆ 2. Leader 将写请求以 Proposal 的形式发给所有 Follower 并等待 ACK
- ◆ 3. Follower 收到 Leader 的 Proposal 后返回 ACK
- ◆ 4. Leader 得到过半数的 ACK (Leader 对自己默认有一个 ACK) 后向所有的 Follower 和 Observer 发送 Commit
- ◆ 5. Leader 将处理结果返回给客户端



Zookeeper Misc.

◆ 一致性：写 Follower 场景数据一致性

- ◆ 1. 客户端向 Follower 发起写请求，Follower 将写请求转发给 Leader 处理
- ◆ 2. 其它流程与直接写 Leader 无任何区别



Zookeeper Misc.

◆ 一致性：最终一致性

◆ 数据写入过程核心分成下面两阶段：

- ◆ 第一阶段：Leader数据写入事件作为提案广播给所有Follower 结点；可以写入的Follower结点返回确认信息ACK
- ◆ 第二阶段：Leader 收到一半以上的 ACK 信息后确认写入可以生效，向所有结点广播 COMMIT 将提案生效

◆ 根据写入过程的两阶段的描述，可以知道 Zookeeper 保证的是最终一致性

- ◆ 即 Leader 向客户端返回写入成功后，可能有部分Follower 还没有写入最新的数据，所以是最终一致性

Zookeeper Misc.

◆ 一致性：最终一致性

- ◆ Zookeeper 保证的最终一致性也叫顺序一致性，即每个结点的数据都是严格按事务的发起顺序生效的
 - ◆ ZK事务的顺序性是借助上文中的Zxid实现的
- ◆ Leader 在收到半数以上 ACK 后会将提案生效并广播给所有 Follower 结点，Leader 为了保证提案按 ZXID 顺序生效，使用了一个 ConcurrentHashMap，记录所有未提交的提案，命名为 **outstandingProposals**，key 为 ZXID，Value 为提案的信息

Zookeeper Misc.

◆ 一致性：最终一致性

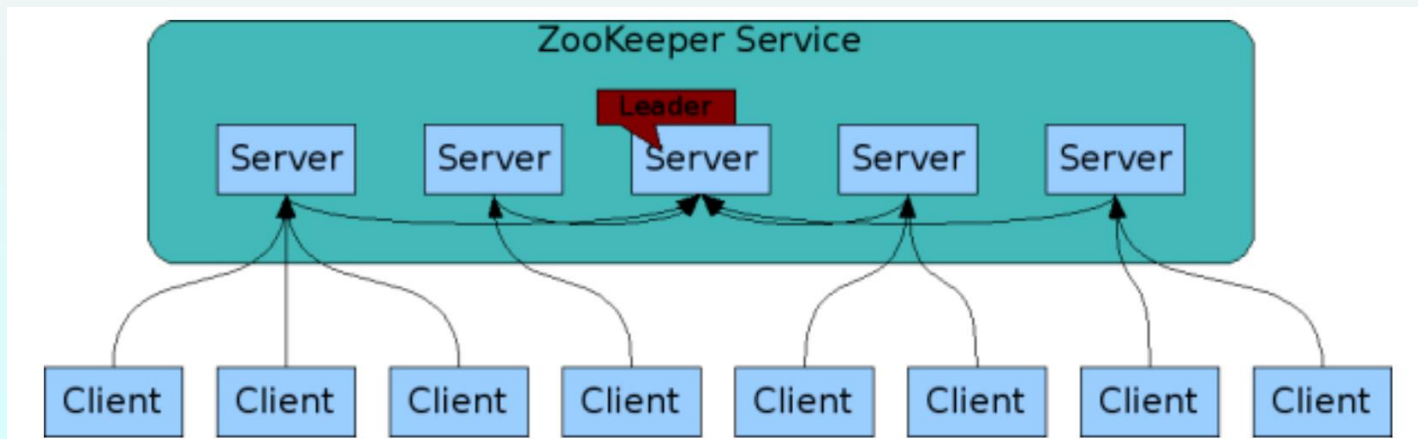
- ◆ 对 outstandingProposals 的访问逻辑如下
- ◆ 1. Leader 每发起一个提案，会将提案的 ZXID 和内容放到 outstandingProposals 中，作为待提交的提案；
- ◆ 2. Leader 收到 Follower 的 ACK 信息后，根据 ACK 中的 ZXID 从 outstandingProposals 中找到对应的提案，对 ACK 计数；
- ◆ 3. 执行 tryToCommit 尝试将提案提交：判断流程是，先判断当前 ZXID 之前是否还有未提交提案，如果有，当前提案暂时不能提交；再判断提案是否收到半数以上 ACK，如果达到半数则可以提交；如果可以提交，将当前 ZXID 从 outstandingProposals 中清除，并向 Followers 广播提交当前提案
 - ◆ Leader 是如何判断当前 ZXID 之前是否还有未提交提案的呢？由于前提是保证顺序提交的，所以 Leader 只需判断 outstandingProposals 里，当前 ZXID 的前一个 ZXID 是否存在
- ◆ 总结，ZooKeeper 是通过两阶段提交保证数据的最终一致性
- ◆ 并且通过严格按照 ZXID 的顺序生效提案保证其顺序一致性的

pause



Zookeeper Misc.

- ◆ **Zookeeper架构**: 为了确保服务的高可用性, ZK采用集群化部署
 - ◆ Zookeeper分为服务器端(Server)和客户端(Client)
 - ◆ 客户端可以连接到整个Zookeeper服务的任意服务器上 (Leader除外)
 - ◆ Zookeeper启动时, 将从实例中选举一个Leader, Leader负责处理数据更新等操作



Zookeeper Misc.

◆ Zookeeper中的四种角色

- ◆ (1) **Leader**: 领导者, 一个 ZK 集群同一时间只会有一个实际工作的 Leader; 所有的写操作必须要通过 Leader 完成再由 Leader 将写操作广播给其它服务器
- ◆ (2) **Follower(Learner的子类)**: 跟随者, 一个 ZK 集群可同时存在多个 Follower; 用于Follower可以接收Client请求, 直接处理并返回客户端的读请求, 同时会将写请求转发给 Leader 处理; 参与事务请求Proposal的投票及Leader选举投票
- ◆ (3) **Observer**: 观察者, Observer是3.3.0 版本开始引入的一个服务器角色, 一个 ZK 集群可同时存在多个 Observer; 可以接收客户端连接, 将写请求转发给Leader节点, **但是不参与投票过程**, 只是同步Leader状态, 因为Follower增多会导致投票阶段延迟增大, 影响性能
- ◆ Observer的目的是为了扩展系统, 提高读取数据
- ◆ (4) **Learner**: 学习者

pause



Zookeeper Misc.

- ◆ **Leader的选举：** 准备知识
- ◆ (1) Zookeeper中的Server数目一般为奇数？
 - ◆ 我们知道在Zookeeper中 Leader 选举算法采用了Quorum算法；该算法的核心思想是当多数Server写成功，则任务数据写成功
 - ◆ 假设有3个Server，则最多允许一个Server挂掉；如果有4个Server，则同样最多允许一个Server挂掉
 - ◆ 既然3个或者4个Server，同样最多允许1个Server挂掉，那么它们的可靠性是一样的，所以选择奇数个Zookeeper Server即可，这里选择3个Server

Zookeeper Misc.

- ◆ **Leader的选举：** 准备知识
- ◆ **(2) Quorum算法**
 - ◆ 在分布式系统中，冗余数据是保证可靠性的手段，因此冗余数据的一致性维护就非常重要
 - ◆ 一般而言，一个写操作必须要对所有的冗余数据都更新完成了，才能称为成功结束
 - ◆ 比如一份数据在5台设备上有冗余，因为不知道读数据会落在哪一台设备上，那么一次写操作，必须5台设备都更新完成，写操作才能返回
 - ◆ 对于写操作比较频繁的系统，这个操作的瓶颈非常大；Quorum算法可以让写操作只要写完3台就返回；剩下的由系统内部缓慢同步完成
 - ◆ 而读操作，则需要也至少读3台，才能保证至少可以读到一个最新的数据

Zookeeper Misc.

- ◆ **Leader的选举：** 准备知识
- ◆ **(3) Zookeeper节点的一些状态**
 - ◆ **1) LOOKING:** 寻找Leader状态；当服务器处于该状态时，它会认为当前集群中没有 Leader，因此需要进入 Leader 选举状态
 - ◆ **2) LEADING:** 说明此节点已经是leader节点，处于领导者地位的状态，就是一般集群中的master；在zookeeper中，只有leader才有写权限，其他节点（FOLLOWING）是没有写权限的，可以读
 - ◆ **3) FOLLOWING:** 跟随者状态，表示当前集群中的leader已经选举出来了，本台服务器处于跟随状态；同步 leader 状态，参与投票
 - ◆ **4) OBSERVING:** OBSERVING和FOLLOWING差不多，同步 leader 状态，不参与投票

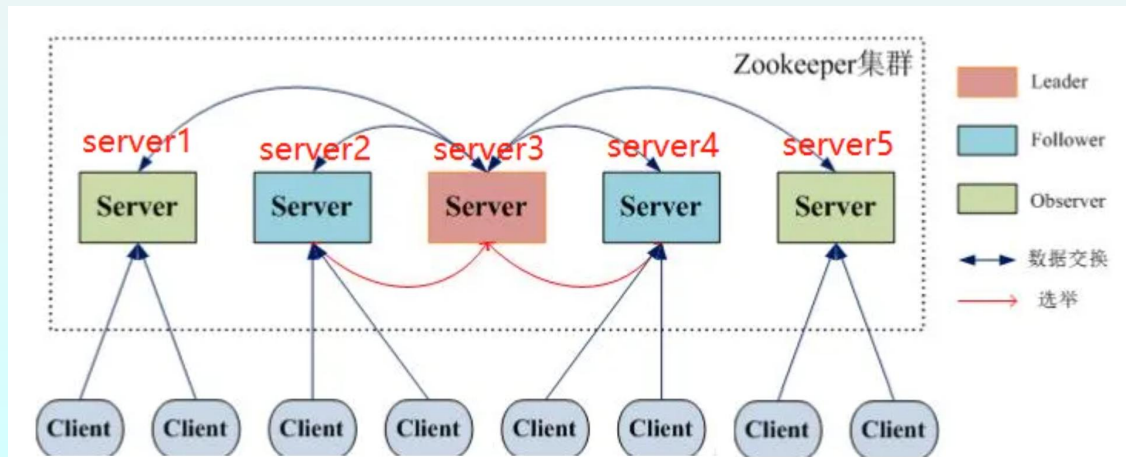
pause



Zookeeper Misc.

◆ Leader的选举过程：准备

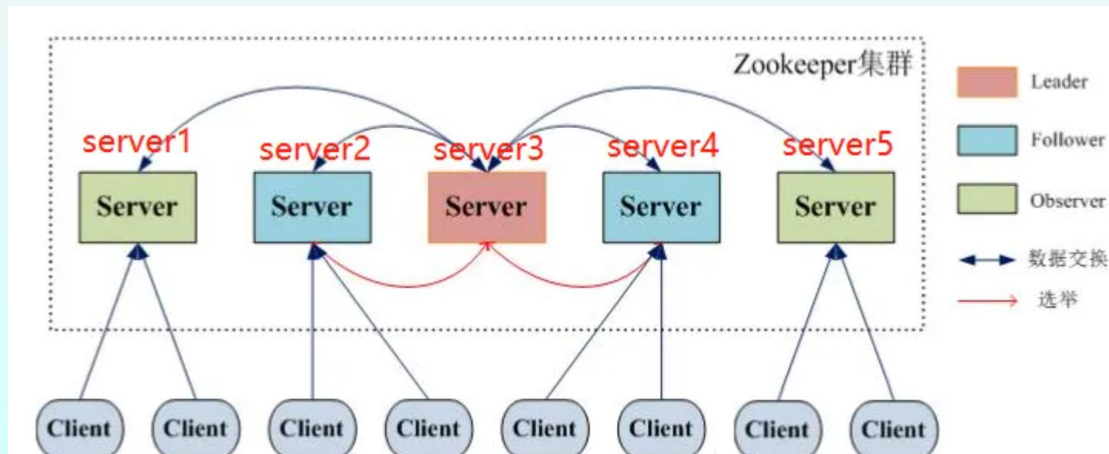
- ◆ 1.每个Server都有一个唯一标识自己的myid；选举过程中每个Server都有一票（挂掉的除外）；在这里，从左到右分别将五台服务器的myid赋值为1、2、3、4、5



Zookeeper Misc.

◆ Leader的选举过程：准备

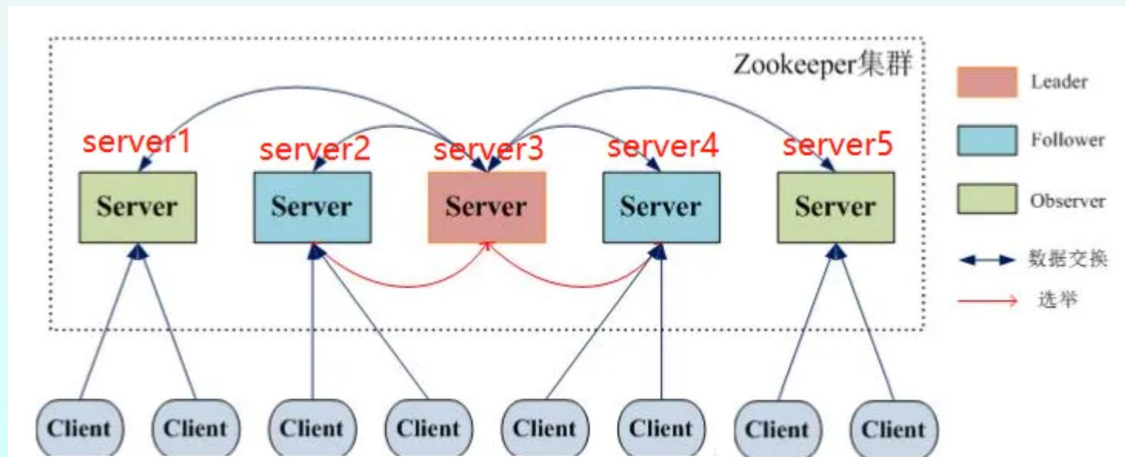
- ◆ 2. zxid类似于RDBMS中的事务ID，用于标识一次更新操作的Proposal ID；每次请求对应一个唯一的zxid,如果zxid $a < \text{zxid } b$, 则可以保证a一定发生在b之前
- ◆ Zookeeper为了保证数据的有序性，给每个写操作的数据，编写一个全局唯一的zxid
- ◆ 先生成的zxid较小，后生成的zxid大，后存储的数据的zxid一定大于先存储的数据的zxid



Zookeeper Misc.

◆ Leader的选举过程：准备

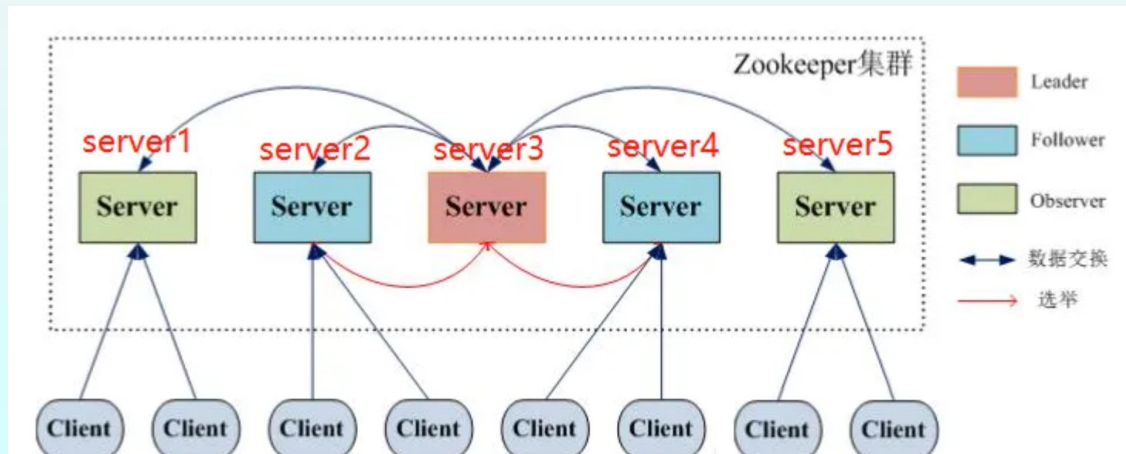
- ◆ 3. 选举分为两种情况：初始化的时候（还没有leader）； leader服务器挂掉的时候（重新选举新的leader）



Zookeeper Misc.

◆ Leader的选举过程：选举3原则

- ◆ (1) zookeeper集群中只有超过半数以上的服务器启动，此集群才能正常工作
- ◆ (2) 集群正常工作之前，myid小的服务器会给myid大的服务器投票，投票一直持续到集群开始正常工作，即选出leader
- ◆ (3) 选出leader以后，leader之前的服务器节点状态要由looking，转为following，leader之后的节点、新加入节点变成following



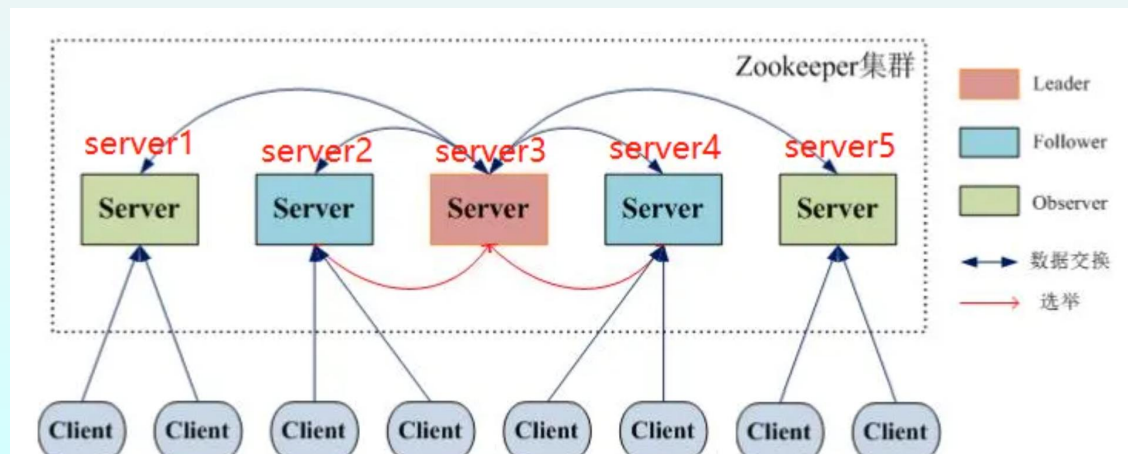
pause



Zookeeper Misc.

◆ Leader的选举过程：初始化的情况

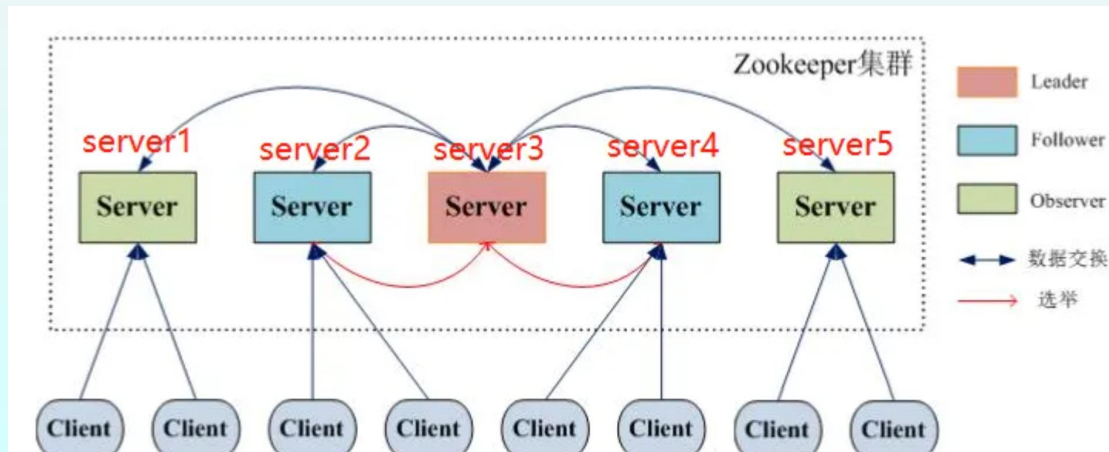
- ❖ (1) 首先第一台服务器启动，投自己一票，然后发投票信息，由于其它机器还没有启动所以它收不到反馈信息，服务器1的状态一直属于**Looking**
- ❖ (2) 服务器2启动，发现当前没有leader，投票给自己，同时与之前启动的服务器1交换结果，由于服务器2的编号大所以服务器2胜出，但此时投票数没有大于半数，所以两个服务器的状态依然是**LOOKING**



Zookeeper Misc.

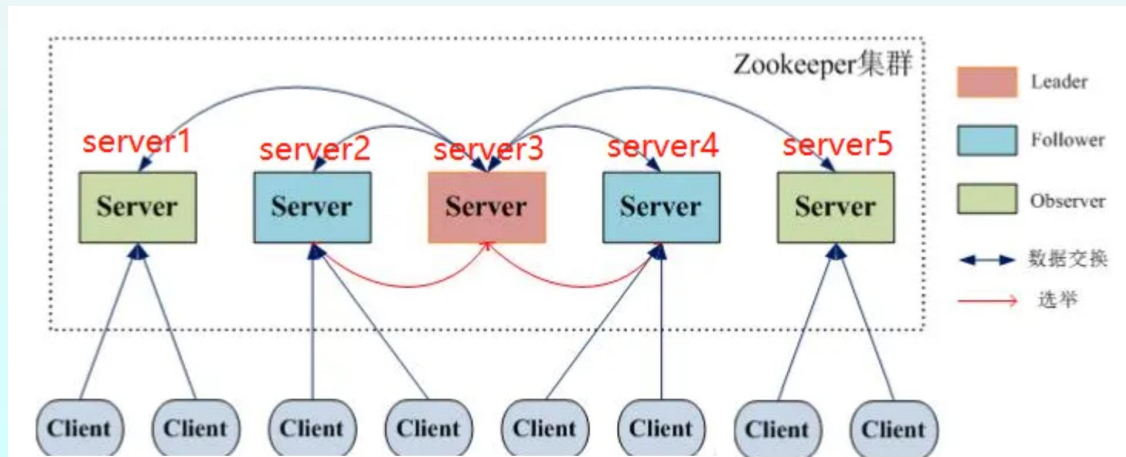
◆ Leader的选举过程：初始化的情况

- ◆ (3) 服务器3启动，发现没有Leader，给自己投票，同时与之前启动的服务器1、2交换信息，由于服务器3的编号最大所以服务器3胜出，此时投票数正好大于半数，所以服务器3成为领导者，服务器1、2成为小弟
- ◆ (4) 服务器4启动，给自己投票，同时与之前启动的服务器1、2、3交换信息，尽管服务器4的编号大，但之前服务器3已经胜出，所以服务器4只能成为小弟



Zookeeper Misc.

- ◆ Leader的选举过程：初始化的情况
 - ◆ (5) 服务器5和服务器4逻辑相同



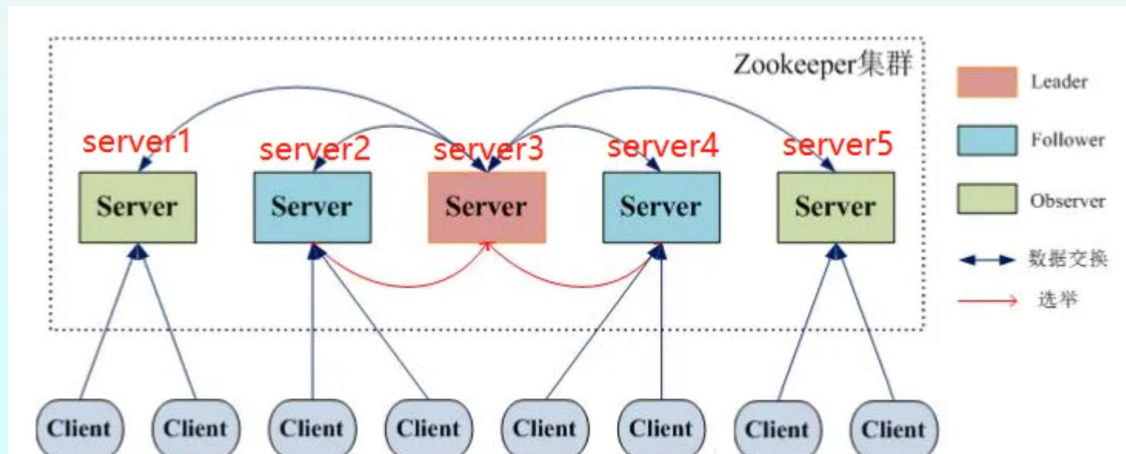
pause



Zookeeper Misc.

◆ Leader的选举过程：重新选举Leader

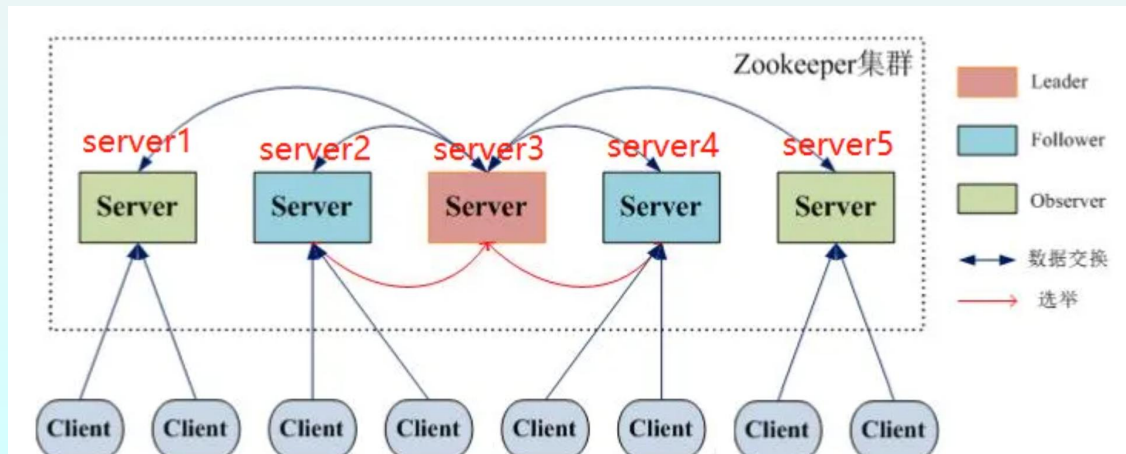
- ◆ 如果Server3当选了Leader，它突然宕机了，那么就需要重新选Leader
- ◆ (1) Leader宕机之后，剩下的机器会自动进入选举状态，重新选举
- ◆ (2) 选举的依据是：优先考虑数据的版本号zxid，再考虑服务器id(因为zxid越大，代表该服务器的数据越新，越全)



Zookeeper Misc.

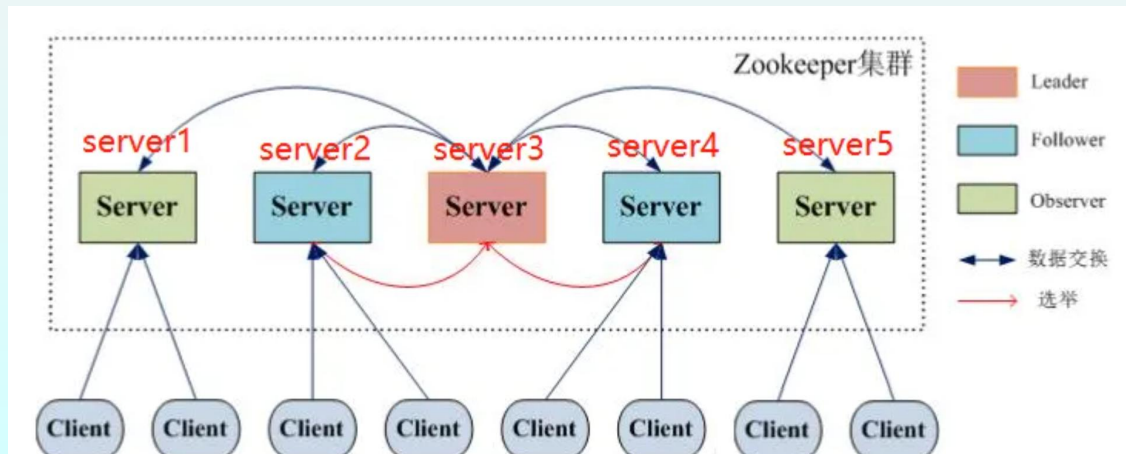
◆ Leader的选举过程：重新选举Leader

- ◆ (3) 由于是运行期间，因此每个服务器上的ZXID可能不同我们假设Server1的ZXID为123，而Server2的ZXID为122，Server4的ZXID为143，Server5的ZXID为120
- ◆ (4) 在第一轮投票中，Server1、Server2、Server4、Server5都会投给自己，即分别产生投票(1,123)，(2,122)，(4,143)，(5,120)



Zookeeper Misc.

- ◆ **Leader的选举过程：重新选举Leader**
 - ◆ (5) 然后各自将这个投票发给集群中所有机器
 - ◆ (6) 对于投票的处理，在这个情境下Server4的ZXID为143,是最大的，显然Server4会成为Leader，其他服务器成为跟随者



pause



Zookeeper Misc.

◆ Zookeeper应用及其描述：数据发布/订阅

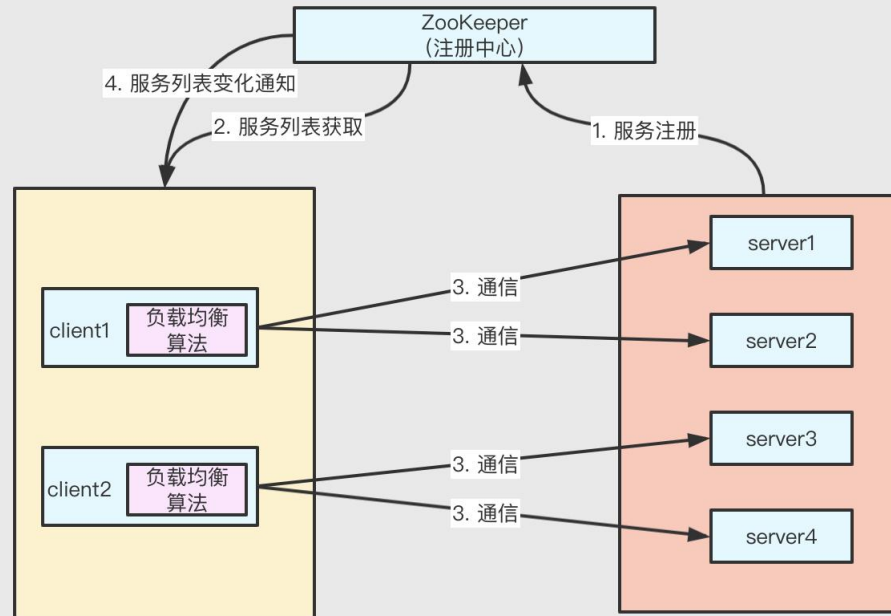
- ◆ ZK基于Watcher监听机制实现数据的发布与订阅功能
- ◆ 发布订阅功能可以看作是一个一对多的关系，即一个服务或数据的发布者可以被多个不同的消费者调用
- ◆ 一般一个发布订阅模式的数据交互可以分为消费者主动请求生产者信息的拉取模式，和生产者数据变更时主动推送给消费者的推送模式
- ◆ zk的发布订阅模式采用的是推拉结合的方式实现的



Zookeeper Misc.

◆ Zookeeper应用及其描述：负载均衡

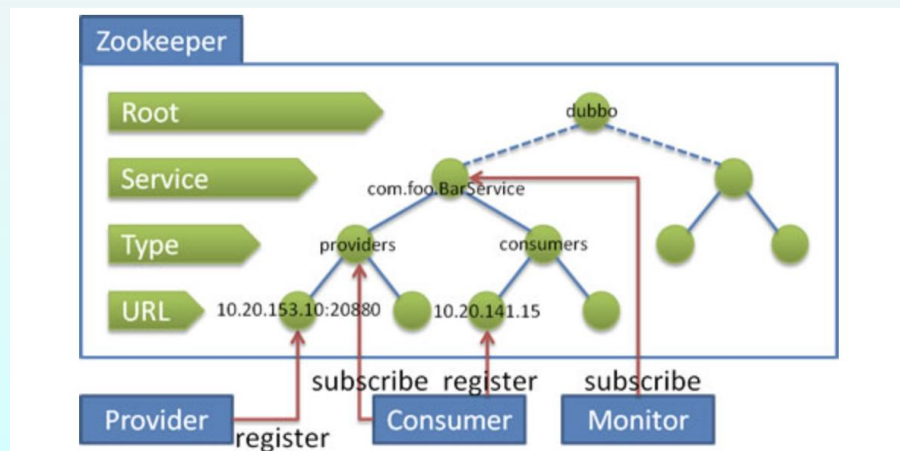
- ◆ 利用ZK的临时节点、watcher机制等特性可实现负载均衡；把ZK作为一个服务的注册中心，基本流程
- ◆ 1. 服务提供者server启动时在ZK进行服务注册（创建临时文件）
- ◆ 2. 服务消费者client启动时，请求ZK获取最新的服务存活列表并注册watcher，然后将获得服务列表保存到本地缓存中
- ◆ 3. client请求server时，根据自己的负载均衡算法，从服务器列表选取一个进行通信
- ◆ 4. 若在运行过程中，服务提供者出现异常或人工关闭不能提供服务，临时节点失效，ZK探测到变化更新本地服务列表并异步通知到服务消费者，服务消费者监听到服务列表的变化，更新本地缓存



Zookeeper Misc.

◆ Zookeeper应用及其描述：命名服务

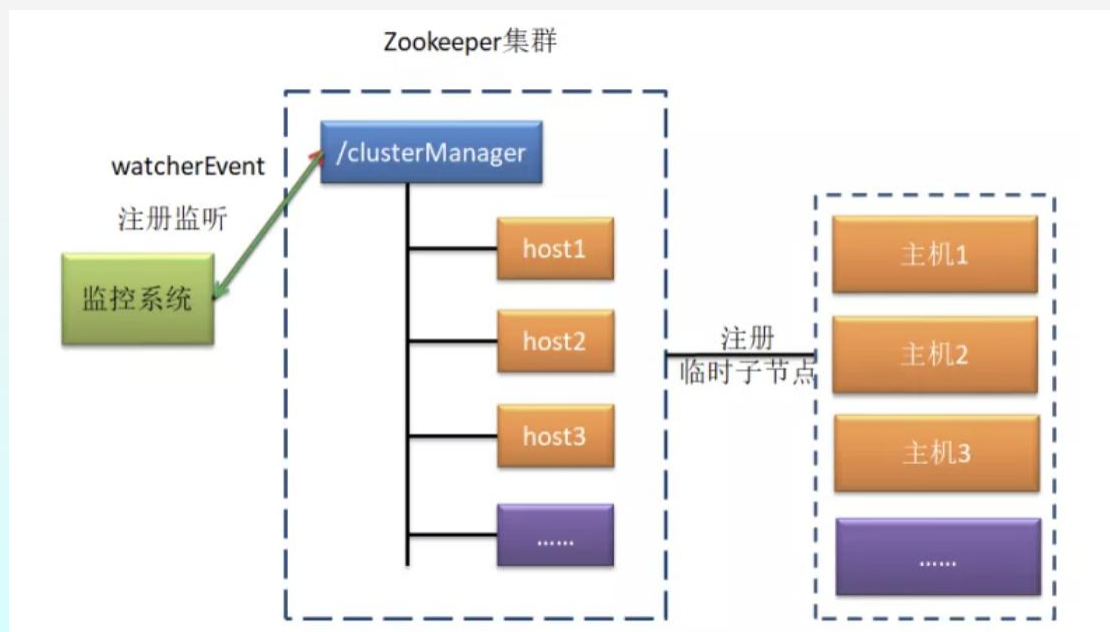
- ◆ 命名服务是指通过指定的名字来获取资源或者服务的地址、提供者等信息。利用ZK很容易创建一个全局的路径，而这个路径就可以作为一个名字，它可以指向集群、集群提供的服务的地址、远程对象等
- ◆ 例如Dubbo使用ZK来作为其命名服务，所有 Dubbo 相关的数据都组织在 /dubbo 的根节点下；二级目录是服务名，如 `com.foo.BarService`；三级目录有两个子节点，分别是 `providers` 和 `consumers`，表示该服务的提供者和消费者。四级目录记录了与该服务相关的每一个应用实例的 URL 信息，在 `providers` 下的表示该服务的所有提供者，而在 `consumers` 下的表示该服务的所有消费者



Zookeeper Misc.

◆ Zookeeper应用及其描述： 集群管理

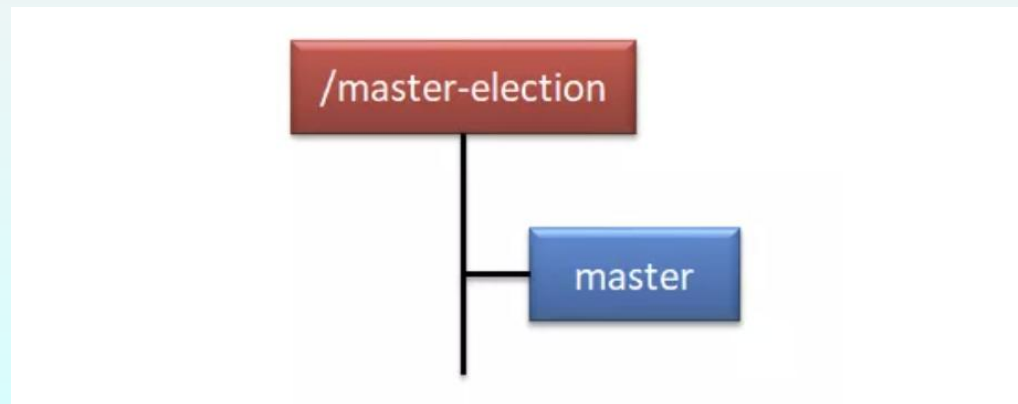
- ◆ 集群管理通常指监控集群中各个主机的运行时状态、存活状况等信息。通过 zk 可以实现对集群的随时监控。其基本原理是使用zk的临时节点来进行监控
- ◆ 如下图所示，主机向zk注册临时节点，监控系统注册监听集群下的临时节点，从而获取集群中服务的状态等信息



Zookeeper Misc.

◆ Zookeeper应用及其描述: Master 选举

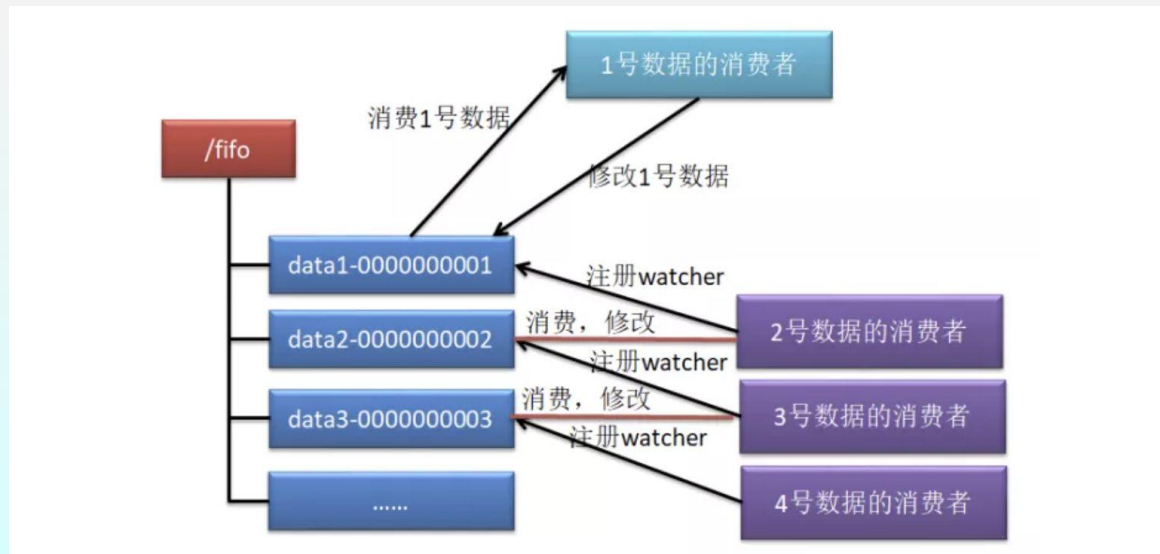
- ◆ zk中多个客户端创建同一节点时，只有一个客户端可以成功。利用该特性，可以实现maser选举
- ◆ 1. 多个客户端同时竞争创建同一临时节点/master-election/master，最终只能有一个客户端成功。这个成功的客户端成为Master，其它客户端置为Slave。 2. Slave客户端都向这个临时节点的父节点/master-election 注册一个子节点列表的watcher 监听。 3. 一旦原Master宕机，临时节点就会消失，zk 服务器就会向所有 Slave 发送子节点变更事件，Slave 在接收到事件后会竞争创建新的master临时子节点。谁创建成功，谁就是新的 Master



Zookeeper Misc.

◆ Zookeeper应用及其描述： 分布式队列

- ◆ 基于ZK的临时顺序节点和Watcher 机制可实现简单的FIFO分布式队列
- ◆ zk 实现 FIFO 队列的思路是： 利用顺序节点的有序性， 为每个数据在 zk 中都创建一个相应的节点
- ◆ 然后为每个节点都注册 watcher 监听
- ◆ 一个节点被消费， 则会引发消费者消费下一个节点， 直到消费完毕



Zookeeper Misc.

◆ Zookeeper应用落地

◆ Kafka

- ◆ ZK在Kafka 集群中扮演着极其重要的角色，负责包括 broker集群信息维护、 topic相关信息的注册/订阅、Consumer的 offset 记录（随着Kafka的更新，此处已不再依赖ZK）、partition选主等功能

◆ Dubbo

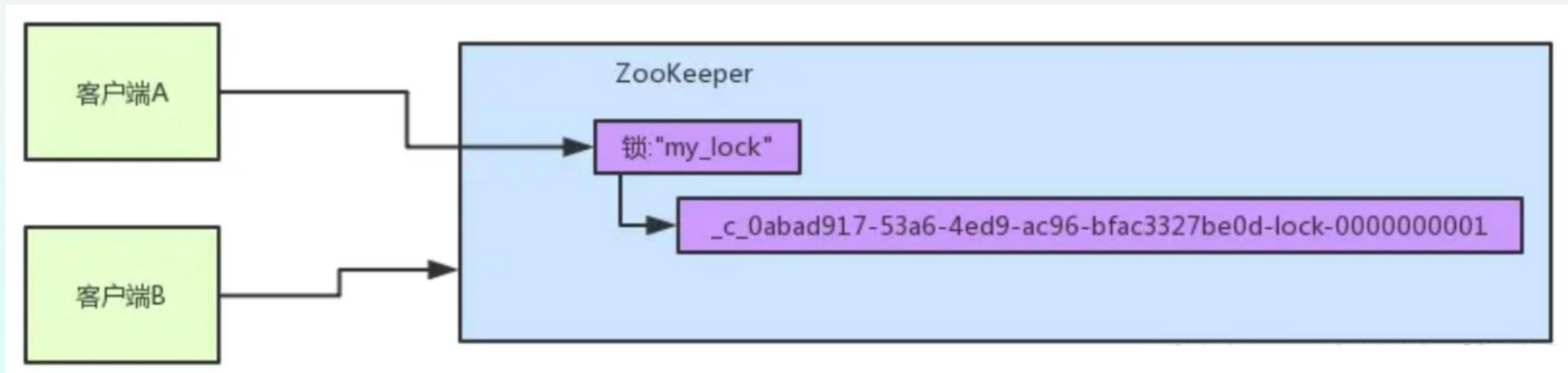
- ◆ Dubbo 使用 Zookeeper 用于服务的注册发现和配置管理，详见前文

pause



Zookeeper Misc.

- ◆ **Zookeeper应用及其描述：** 分布式锁
 - ◆ 基于ZK的**临时顺序节点**和**Watcher 机制**可实现公平分布式锁
 - ◆ 假如客户端A先发起请求，它会建立一个顺序节点，如下图



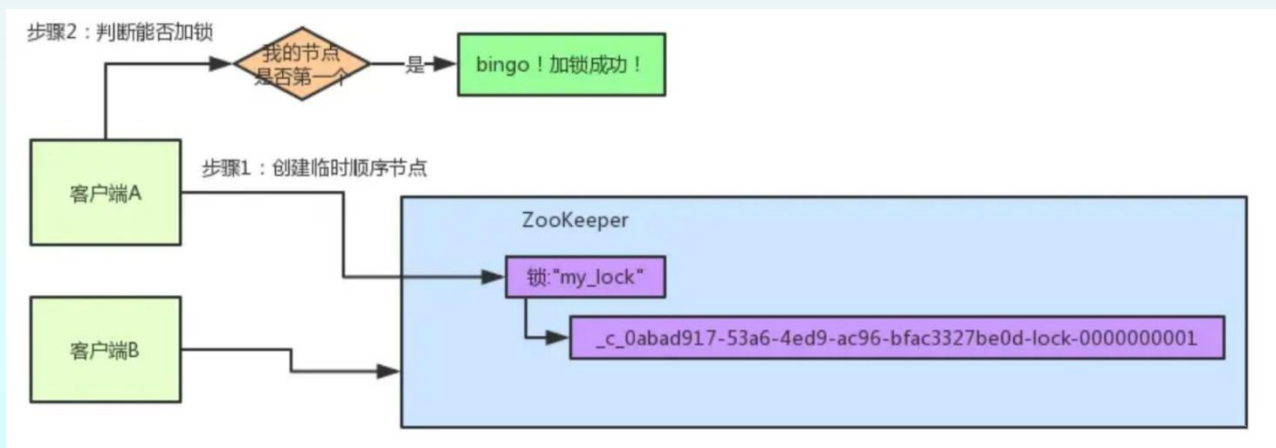
Zookeeper Misc.

- ◆ **Zookeeper应用及其描述：** 分布式锁
 - ◆ 因为客户端A是第一个发起请求的，所以给它建立的顺序节点的序号是"1"
 - ◆ 接着客户端A查一下“my_lock”这个锁节点下的所有子节点，这些子节点是按照序号排序的，这个时候它会拿到这么一个集合

```
[  
  "_c_0abad917-53a6-4ed9-ac96-bfac3327be0d-lock-0000000001"  
]
```

Zookeeper Misc.

- ◆ **Zookeeper应用及其描述：** 分布式锁
 - ◆ 客户端A进行判断，这个集合里，它创建的那个顺序节点，是不是排在第一个啊？
 - ◆ 如果是的话，那就可以加锁了！确实它就是第一个来创建顺序节点的人，所以它就是第一个尝试加分布式锁的人啊！
 - ◆ 现在加锁成功！如下图所示



Zookeeper Misc.

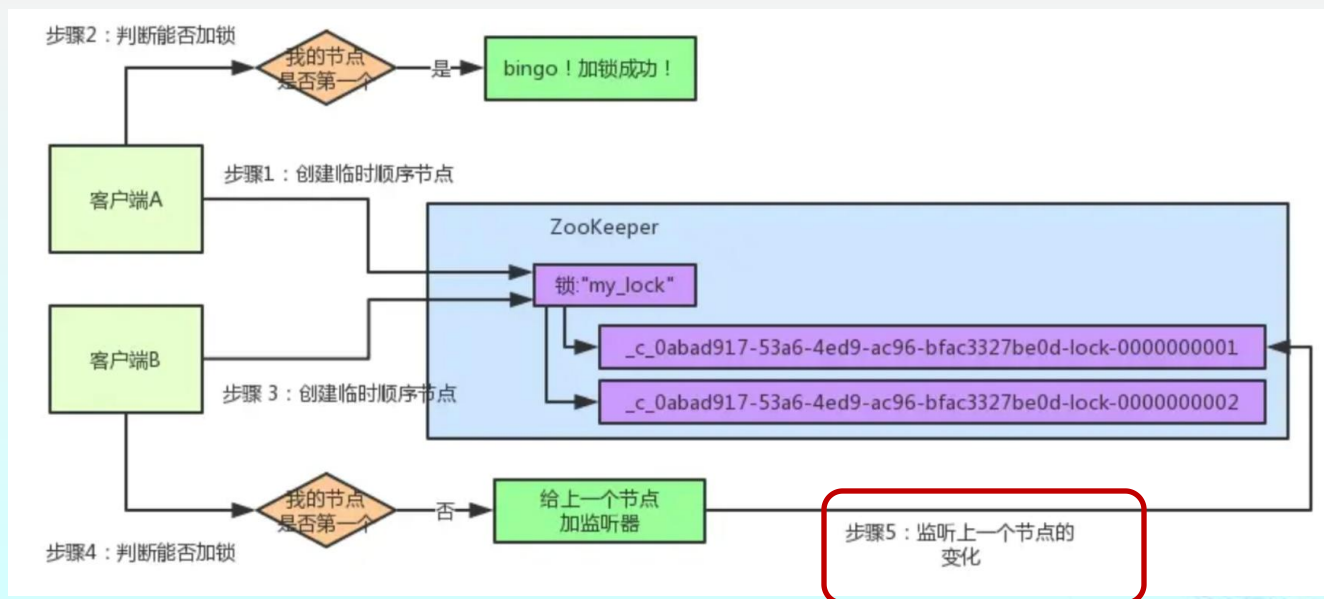
- ◆ **Zookeeper应用及其描述：** 分布式锁
 - ◆ 假如客户端A加完锁完后，客户端B过来想要加锁，这个时候它先在"my_lock"这个锁节点下创建一个临时顺序节点，因为是第二个来创建顺序节点的，所以zk内部会维护序号为"2"
 - ◆ 接着客户端B会执行加锁判断逻辑，查询“my_lock”锁节点下的所有子节点，按序号顺序排列，这时候它看到

```
[  
  "_c_0abad917-53a6-4ed9-ac96-bfac3327be0d-lock-0000000001",  
  "_c_0abad917-53a6-4ed9-ac96-bfac3327be0d-lock-0000000002"  
]
```

Zookeeper Misc.

◆ Zookeeper应用及其描述： 分布式锁

- ◆ 检查自己创建的顺序节点，是不是集合中的第一个？明显不是，此时第一个是客户端A创建的那个顺序节点，序号为“01”的那个，所以加锁失败！
- ◆ 加锁失败了以后，客户端B就会通过ZK的API对它的顺序节点的上一个顺序节点加一个监听器，即对客户端A创建的那个顺序节加监听器



Zookeeper Misc.

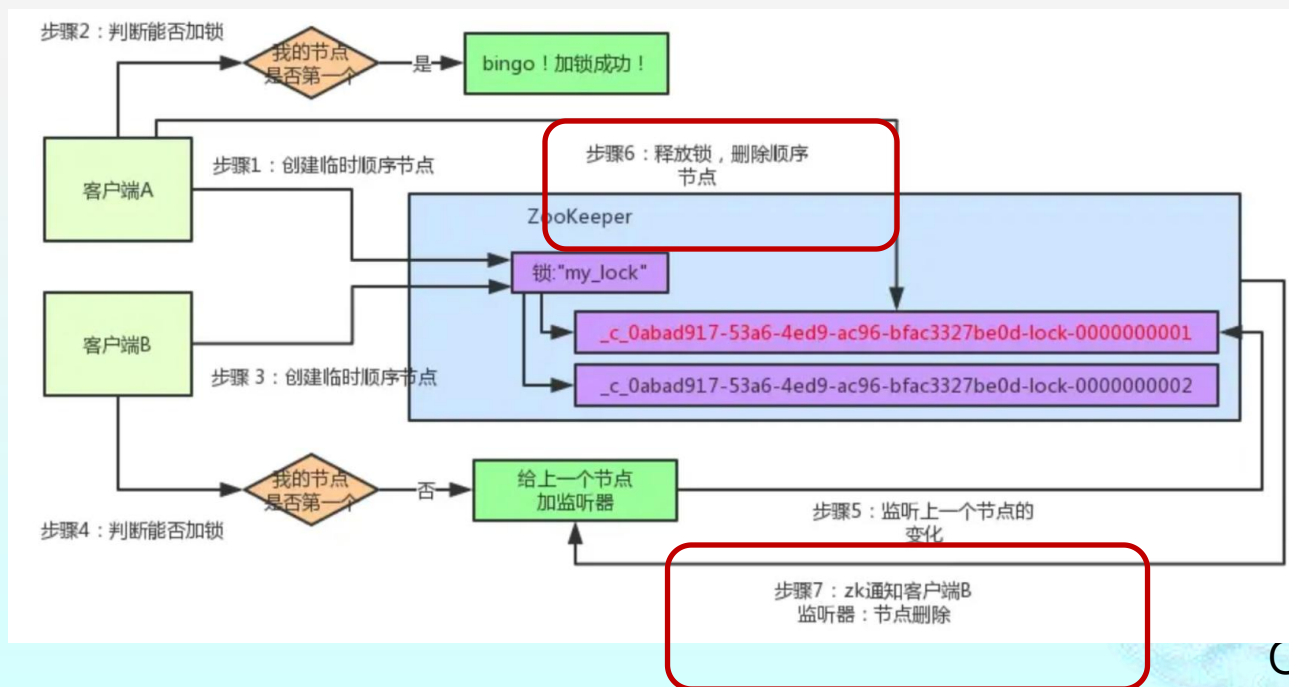
- ◆ **Zookeeper应用及其描述：** 分布式锁
 - ◆ 客户端A加锁之后，可能处理了一些代码逻辑，然后就会释放锁。
 - ◆ 释放锁是个什么过程呢？很简单，就是把自己在zk里创建的那个顺序节点删除

```
[
  "_c_0abad917-53a6-4ed9-ac96-bfac3327be0d-lock-0000000001",
  "_c_0abad917-53a6-4ed9-ac96-bfac3327be0d-lock-0000000002"
]
```

Zookeeper Misc.

◆ Zookeeper应用及其描述： 分布式锁

- ◆ 删除了那个节点之后，zk会负责通知监听这个节点的监听器
- ◆ 也就是客户端B之前加的那个监听器，说你监听的那个节点被删除了，有人释放了锁



Zookeeper Misc.

◆ Zookeeper应用及其描述： 分布式锁

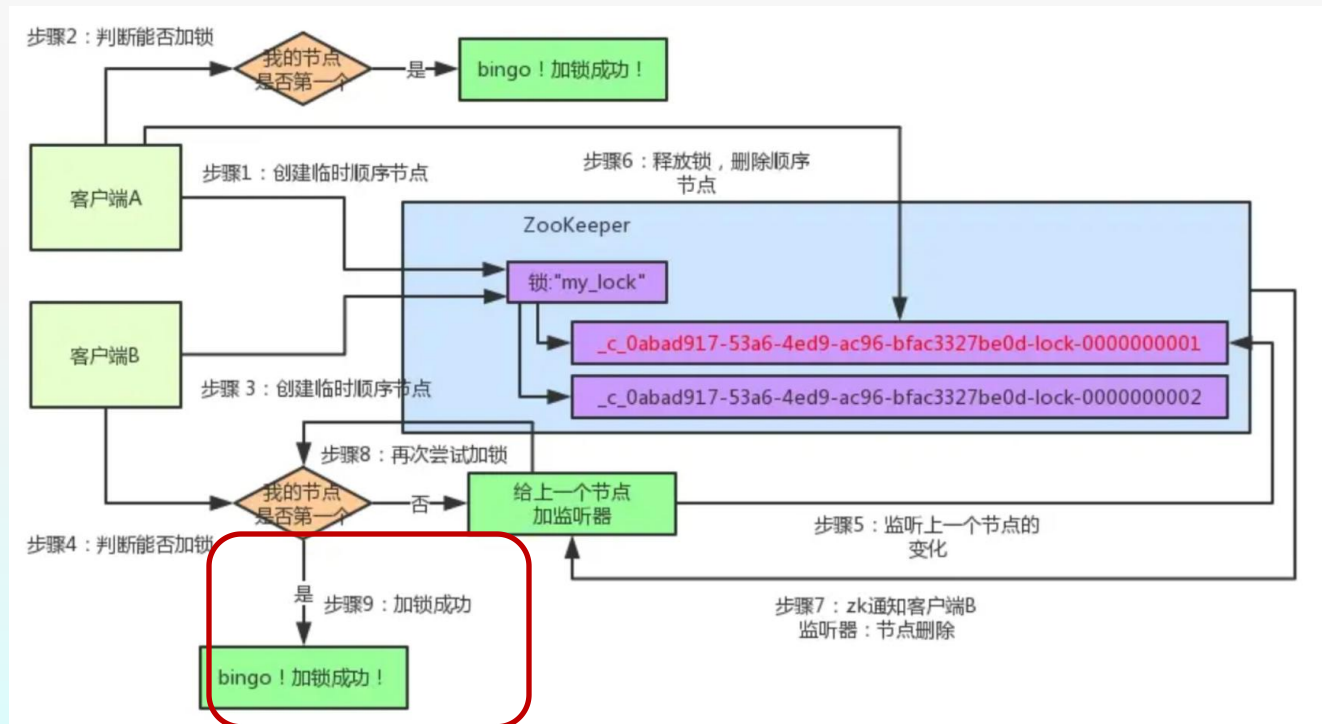
- ◆ 此时客户端B的监听器感知到了上一个顺序节点被删除，也就是排在它之前的某个客户端释放了锁
- ◆ 此时，就会通知客户端B重新尝试去获取锁，也就是获取“my_lock”节点下的子节点集合，具体如下，集合里此时只有客户端B创建的唯一的一个顺序节点了！

```
[  
  _c_0abad917-53a6-4ed9-ac96-bfac3327be0d-lock-0000000002  
]
```

- ◆ 客户端B判断自己目前是集合中的第一个顺序节点，可以加锁了！
- ◆ 完成加锁，运行后续的业务代码，运行完了之后再次释放锁

Zookeeper Misc.

◆ Zookeeper应用及其描述： 分布式锁



pause

